CS-590 Database Design and Development

7-1 Assignment: Additional Integration Using Jupyter Notebook

Malgorzata Debska

Southern New Hampshire University

Zach Probst

March 16, 2026

Part Three: Reflection on the ETL Process

In modern data-driven environments, applications often depend on reliable methods for moving and transforming data between systems. One widely used approach for managing data integration is the extract, transform, and load process. ETL refers to the process of extracting data from a source system, transforming it into a structure or format that meets the requirements of a destination system, and loading the transformed data into a database or data warehouse. ETL processes are essential for ensuring that data is still exact, consistent, and accessible for analysis and operational use (Kimball & Caserta, 2011). In this assignment, Python code was used within Jupyter Notebook to integrate with MongoDB and Neo4j databases. Python can also be used to implement ETL pipelines when transferring data between systems or preparing datasets for storage and analysis.

One circumstance that could lead to using Python code to perform ETL processes is when data must be migrated between different database systems or when the structure of the data needs to be changed before being stored. For example, a dataset originally stored in a relational database or external file such as CSV or JSON may need to be imported into MongoDB or Neo4j. Python scripts can extract the data from the source system, transform the data structure to match the schema or structure required by the destination database, and then load the processed data into the new environment. This approach is particularly useful when working with heterogeneous databases such as MongoDB, which is a document-oriented database, and Neo4j, which is a graph database. Each of these database models stores and organizes data differently, meaning that transformation is often required before the data can be successfully loaded and queried.

The complexity of an ETL process can vary depending on the size, structure, and quality of the data being processed. In simple scenarios, ETL may involve extracting data from a file and inserting it directly into a database with minimal transformation. However, in more complex situations, the transformation phase may involve several operations such as filtering irrelevant data, standardizing values, converting data types, restructuring data relationships, or removing duplicate records. For example, when loading nutrient data into MongoDB, the transformation process may involve converting structured tabular data into JSON-style documents that match the structure of MongoDB collections. When loading the same data into Neo4j, added transformations may be needed to stand for relationships between nodes. Nutrient descriptions might be represented as nodes, while relationships between nutrients and food categories might be represented as graph edges. These structural differences increase the complexity of the ETL process and require careful planning to ensure that the transformed data accurately reflects the intended relationships.

The source of the data can also influence the design and complexity of an ETL process. Data that originates from structured sources such as relational databases often have clearly defined schemas and consistent formatting, which simplifies the extraction and transformation stages. In contrast, data collected from external sources such as web APIs, spreadsheets, or user-generated input may hold inconsistencies, missing values, or formatting variations. These inconsistencies require more transformation and cleaning steps before the data can be loaded into a database. For instance, if nutrient data were collected from multiple external providers, Python scripts might need to reconcile differences in column names, standardize measurement units, or correct formatting inconsistencies. The reliability and structure of the source data therefore play a

critical role in finding the overall design and complexity of the ETL pipeline (Kimball & Caserta, 2011).

When implementing ETL processes using Python, several validation techniques can be included to support data quality and integrity. One important method is schema validation, which ensures that incoming records have the required fields and correct data types before they are inserted into the database. For example, nutrient records should include attributes such as nutrient code, nutrient description, and nutrient units. Python code can verify that these attributes exist and are correctly formatted before performing a database insertion. Another important validation step involves detecting duplicate records. Duplicate entries can negatively affect query accuracy and increase storage usage. Python scripts can compare unique identifiers, such as nutrient codes, to figure out whether a record already exists before inserting a new entry.

More validation techniques include verifying acceptable value ranges, confirming that needed fields are not empty, and validating relationships between entities. For example, when inserting data into Neo4j, Python code may confirm that nodes standing for nutrients exist before proving relationships between those nodes and other entities. Logging and exception handling mechanisms can also be incorporated into ETL scripts to record processing errors and track the success or failure of data operations. These validation techniques improve the reliability of the ETL pipeline and help ensure that data stored within the database is still exact and trustworthy.

From a performance perspective, the efficiency of an ETL process depends on the tools used and the characteristics of the destination database. Python is widely used for ETL pipelines because of its extensive ecosystem of libraries that support database connectivity, data transformation, and automation. However, Python-based ETL processes may experience performance limitations

when processing extremely large datasets because transformations are executed at the application level rather than within the database engine. In such cases, database-native bulk loading tools may offer improved performance.

When comparing ETL efficiency between MongoDB and Neo4j, MongoDB often supports faster bulk ingestion of large datasets due to its document-based architecture and support for batch insert operations. MongoDB collections are designed to store JSON-like documents efficiently, making the database well suited for high-volume data ingestion tasks (MongoDB, 2024). In contrast, Neo4j is customized for managing relationships between entities within a graph structure. While this design enables powerful graph queries and relationship analysis, creating nodes and relationships individually may introduce other overheads during ETL processes. As a result, MongoDB may be more efficient for large-scale data ingestion, while Neo4j provides stronger performance advantages when working with highly connected datasets that require complex relationship queries.

In conclusion, Python provides a flexible and powerful environment for implementing ETL processes when integrating applications with databases such as MongoDB and Neo4j. The ETL process may vary in complexity depending on the structure and quality of the source data as well as the requirements of the destination database. Factors such as schema design, validation requirements, and transformation complexity all influence the design of an ETL pipeline. Implementing validation procedures within Python scripts helps ensure that data stays consistent and reliable as it moves through the ETL workflow. While MongoDB may offer faster bulk data ingestion capabilities, Neo4j offers unique advantages for managing and analyzing complex relationships between entities. Understanding these differences allows developers and data

engineers to select the most effective ETL strategy based on the performance needs and analytical goals of a given application.

Screenshots:

MongoDB Database

Neo4j Database

References

Kimball, R., & Caserta, J. (2011). *The data warehouse ETL toolkit: Practical techniques for extracting, cleaning, conforming, and delivering data*. Wiley.

MongoDB. (2024). *MongoDB documentation*. https://www.mongodb.com/docs/